

Reinforcement Learning

Multi-agent Systems & Agentic AI

Lecture Notes

Easter Term 2026

Marta Grzeskiewicz
mg2192@cam.ac.uk

Lectures

1. **Foundations of RL**
MDP, policies, value functions
2. **Policy Gradient Methods**
REINFORCE, baselines, advantage
3. **GRPO**
Group Relative Policy Optimization
4. **Theory**
Variance, bias, convergence
5. **Agentic Alignment**
RLHF as system, safety, failures

Running Thread

Each lecture builds toward a single question:

How do we train an LLM to behave as intended?

Lectures 1–2: classical tools

Lecture 3: GRPO + RLHF bridge

Lecture 4: why it (sometimes) works

Lecture 5: alignment as a system

Lecture 1

Foundations of Reinforcement Learning

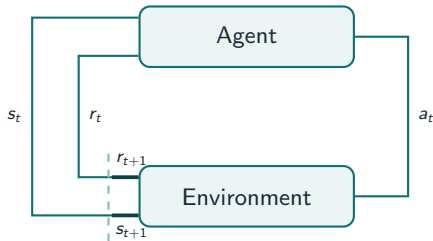
Why Reinforcement Learning?

Supervised learning needs labelled targets.

Many desirable behaviours are **easier to evaluate than to demonstrate**:

- Winning a chess position
- Writing a helpful answer
- Following a user's instruction safely

RL: learn from **scalar feedback** (reward) obtained by acting.



Example: AlphaGo



The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

Symbol	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$P(s' \mid s, a)$	transition kernel
$R(s, a)$	expected reward
$\gamma \in [0, 1)$	discount factor

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

Symbol	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$P(s' s, a)$	transition kernel
$R(s, a)$	expected reward
$\gamma \in [0, 1)$	discount factor

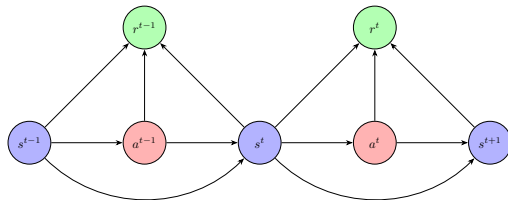


Figure 1: MDP Transition Loop

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

Symbol	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$P(s' s, a)$	transition kernel
$R(s, a)$	expected reward
$\gamma \in [0, 1)$	discount factor

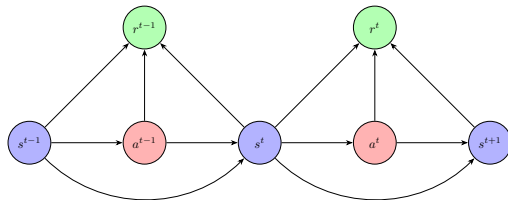


Figure 1: MDP Transition Loop

Markov property

$$P(s_{t+1} | s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} | s_t, a_t)$$

The future is conditionally independent of the past given the present state.

Definition

A **policy** $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions.

Definition

A **policy** $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions.

Stochastic policy

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

Definition

A **policy** $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions.

Stochastic policy

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

Deterministic policy

$$\mu : \mathcal{S} \rightarrow \mathcal{A}$$

- Special case: $\pi(a|s) = \mathbf{1}[a = \mu(s)]$
- Useful for continuous control
- Not directly applicable to LLMs

Definition

A **policy** $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ maps states to distributions over actions.

Stochastic policy

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

Deterministic policy

$$\mu : \mathcal{S} \rightarrow \mathcal{A}$$

- Special case: $\pi(a|s) = \mathbf{1}[a = \mu(s)]$
- Useful for continuous control
- Not directly applicable to LLMs

Later in this course: π_θ is a language model parameterised by weights θ .

The RL Objective

Define the **return** (total discounted reward) from timestep t :

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The RL Objective

Define the **return** (total discounted reward) from timestep t :

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **RL objective** is to maximise:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

The RL Objective

Define the **return** (total discounted reward) from timestep t :

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **RL objective** is to maximise:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

- $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ is a **trajectory**
- $\gamma < 1$ ensures convergence for infinite horizons
- Choice of γ encodes *time preference*

Trajectories and Their Distribution

Trajectory distribution

$$\rho_{\theta}(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t) \pi_{\theta}(a_t \mid s_t)$$

- ρ_0 : initial state distribution
- The policy influences which trajectories are visited
- **Key insight**: gradient of $J(\theta)$ must account for the changing distribution

State-value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state s following π .

Value Functions

State-value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state s following π .

Action-value function

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Expected return taking action a from s , then following π .

Value Functions

State-value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state s following π .

Action-value function

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Expected return taking action a from s , then following π .

Advantage function

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

How much better is action a than the *average* action in state s ?

State-Value Functions

State-value functions $V^\pi(s)$ give the 'value' of state s when following the policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s]$$

State-Value Functions

State-value functions $V^\pi(s)$ give the 'value' of state s when following the policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s]$$

The return (u) can be recursively defined as:

$$\begin{aligned} G_t &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \dots) \\ &= r_t + \gamma G_{t+1} \end{aligned}$$

State-Value Functions

State-value functions $V^\pi(s)$ give the 'value' of state s when following the policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t | S_t = s]$$

The return (u) can be recursively defined as:

$$\begin{aligned} G_t &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \dots) \\ &= r_t + \gamma G_{t+1} \end{aligned}$$

Therefore:

$$V^\pi(s) = \mathbb{E}_\pi [r_t + \gamma G_{t+1} | S_t = s]$$

The Bellman Equation

$$V^\pi(s) = \mathbb{E}_\pi[r_t + \gamma G_{t+1} \mid S_t = s] \quad (1)$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s_t) \sum_{s' \in \mathcal{S}} P(s' \mid s, a) [R(s, a) + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']] \quad (2)$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s, a) [R(s, a) + \gamma V^\pi(s')] \quad (3)$$

The last equation is known as the **Bellman equation** in honour of Richard Bellman.

State-Action Value Function

State-action value functions $Q^\pi(s, a)$ are an extension on the *State* value functions. They condition the expected return on the current state and the action taken:

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t | S_t = s, A_t = a]$$

The *state-action* value Bellman equation is therefore:

$$Q^\pi(s, a) = \sum_{s' \in S} P(s'|s, a) [\mathcal{R}(s, a) + \gamma V^\pi(s')]$$

Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- $A^{\pi}(s, a) > 0 \Rightarrow$ action a is **better than average**; increase its probability

Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- $A^{\pi}(s, a) > 0 \Rightarrow$ action a is **better than average**; increase its probability
- $A^{\pi}(s, a) < 0 \Rightarrow$ action a is **worse than average**; decrease its probability

Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- $A^{\pi}(s, a) > 0 \Rightarrow$ action a is **better than average**; increase its probability
- $A^{\pi}(s, a) < 0 \Rightarrow$ action a is **worse than average**; decrease its probability
- $A^{\pi}(s, a) = 0 \Rightarrow$ indifferent; no update needed

Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- $A^{\pi}(s, a) > 0 \Rightarrow$ action a is **better than average**; increase its probability
- $A^{\pi}(s, a) < 0 \Rightarrow$ action a is **worse than average**; decrease its probability
- $A^{\pi}(s, a) = 0 \Rightarrow$ indifferent; no update needed

Baseline intuition

Subtracting $V^{\pi}(s)$ from $Q^{\pi}(s, a)$ gives a *centered* signal.

This is the single most important variance-reduction idea in RL.

RL as Optimisation over Distributions

Key Reframing

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] = \int p_{\pi_\theta}(\tau) \left(\sum_{t=0}^T \gamma^t r_t \right) d\tau$$

- θ shapes the *distribution* $p_\theta(\tau)$; we optimise over distributions
- This is the fundamental challenge: the expectation depends on θ
- Standard autodiff cannot directly differentiate through *sampling*
- Solution: the **log-derivative trick** (Lecture 2)

Why Not Value Iteration?

Classical **value-based methods** (Q-learning, value iteration):

- Enumerate $\mathcal{S} \times \mathcal{A}$ — infeasible for LLMs ($|\mathcal{A}| \approx 50,000$ tokens, sequences of length $\sim 10^3$)
- Greedy extraction breaks differentiability
- Poor generalisation across states

LLM Scale

A single output sequence is an action.
The action space is combinatorially huge.

Direct policy optimisation with gradient descent is the *only* scalable approach.

Lecture 1 — Summary

Core take-aways

1. RL formalises **sequential decision-making** via MDPs
2. A **policy** $\pi(a|s)$ is what we learn; the **objective** is to maximise the expected discounted return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

3. **Value functions** V^π , Q^π , A^π measure long-run quality
4. The **advantage function** is the key quantity for policy updates
5. RL = optimise a policy over a *distribution of trajectories*

Next: how do we compute $\nabla_\theta J(\theta)$?

Lecture 2

Policy Gradient Methods

The Policy Gradient Objective

We want $\nabla_{\theta} J(\theta)$ where:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)]$$

Challenge: $p_{\theta}(\tau)$ depends on θ , so we cannot simply push ∇_{θ} inside the integral naively.

The Policy Gradient Objective

We want $\nabla_{\theta} J(\theta)$ where:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_{\theta}} [G(\tau)]$$

Challenge: $p_{\theta}(\tau)$ depends on θ , so we cannot simply push ∇_{θ} inside the integral naively.

Log-derivative (REINFORCE) trick

$$\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$$

because $\nabla_{\theta} \log p = \frac{\nabla_{\theta} p}{p}$.

The Log-Derivative Trick

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G(\tau) d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) G(\tau) d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot G(\tau)]\end{aligned}$$

The Log-Derivative Trick

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \nabla_{\theta} \int p_{\theta}(\tau) G(\tau) d\tau \\ &= \int \nabla_{\theta} p_{\theta}(\tau) G(\tau) d\tau \\ &= \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau \\ &= \mathbb{E}_{\tau \sim p_{\theta}} [\nabla_{\theta} \log p_{\theta}(\tau) \cdot G(\tau)]\end{aligned}$$

Since $\log p_{\theta}(\tau) = \log \rho_0(s_0) + \sum_t \log P(s_{t+1}|s_t, a_t) + \sum_t \log \pi_{\theta}(a_t|s_t)$, and only the last term depends on θ :

$$\boxed{\nabla_{\theta} J(\theta) \propto \mathbb{E}_{\tau \sim p_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G(\tau) \right]}$$

The REINFORCE Algorithm

REINFORCE (Williams, 1992)

1. Sample trajectory $\tau \sim \pi_\theta$
2. Compute return $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ for each step t
3. Update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t$$

The REINFORCE Algorithm

REINFORCE (Williams, 1992)

1. Sample trajectory $\tau \sim \pi_\theta$
2. Compute return $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ for each step t
3. Update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t$$

- Unbiased gradient estimate
- No model of P , no value function required

Exploration vs. Exploitation

The core tension

Exploit: repeat actions already known to give high reward.

Explore: try new actions that might be better — but carry risk.

LLM connection

Temperature T controls this trade-off at inference time.

High $T \rightarrow$ flat $\pi_\theta \rightarrow$ exploration.

Low $T \rightarrow$ sharp $\pi_\theta \rightarrow$ exploitation.

How REINFORCE handles it implicitly

- A *stochastic* policy $\pi_\theta(a|s)$ explores by construction — it never deterministically commits to one action.
- As training progresses, high-reward actions accumulate probability mass: the policy becomes **increasingly deterministic**.
- Left unchecked, this causes **premature convergence**: the agent stops exploring before reaching the global optimum.

Code: REINFORCE — Policy and Episode

```
1 import torch, torch.nn as nn
2
3 class Policy(nn.Module):
4     def __init__(self, n_obs, n_act):
5         super().__init__()
6         self.net = nn.Sequential(
7             nn.Linear(n_obs, 64),
8             nn.ReLU(),
9             nn.Linear(64, n_act),
10        )
11
12    def forward(self, obs):
13        logits = self.net(obs)
14        # stochastic policy = Categorical
15        return torch.distributions.Categorical(
16            logits=logits
17        )
18
19 def collect_episode(env, policy):
20     obs = env.reset()
21     log_probs, rewards = [], []
22     done = False
23     while not done:
24         dist = policy(torch.tensor(obs, dtype=torch.float))
25         action = dist.sample()
26         log_probs.append(dist.log_prob(action))
27         obs, r, done, _ = env.step(action.item())
28         rewards.append(r)
29     return log_probs, rewards
```

Key lines:

- `Categorical(logits=)` is $\pi_{\theta}(a|s)$
- `dist.sample()` — stochastic action
- `dist.log_prob(a)` — the score function $\nabla_{\theta} \log \pi$ will come from autodiff on this
- No value network, no critic

Code: REINFORCE — Update Step

```
1 def reinforce_update(log_probs, rewards,
2                       optimizer, gamma=0.99):
3     # 1. Discounted returns
4     G, returns = 0.0, []
5     for r in reversed(rewards):
6         G = r + gamma * G
7         returns.insert(0, G)
8     returns = torch.tensor(returns, dtype=torch.float)
9
10    # 2. Standardise (baseline = mean return)
11    returns = (returns - returns.mean()) \
12              / (returns.std() + 1e-8)
13
14    # 3. Policy gradient loss
15    log_probs_t = torch.stack(log_probs)
16    loss = -(log_probs_t * returns).sum()
17
18    # 4. SGD step
19    optimizer.zero_grad()
20    loss.backward()
21    optimizer.step()
22    return loss.item()
```

What to watch when you run this:

- loss fluctuates wildly — this *is* the variance problem
- Gradient norms vary by $10\times$ across episodes
- Convergence is slow: hundreds of episodes for CartPole

Reference

Spinning Up:

spinningup.openai.com

Clean minimal PyTorch implementations of REINFORCE → PPO.

The Variance Problem

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}$$

Sources of variance:

- Returns G_t accumulate noise over the whole trajectory
- Stochastic transitions multiply uncertainty
- Long horizons: variance scales as $O(T^2)$

Consequence

In practice REINFORCE needs thousands of samples and struggles.

Baselines for Variance Reduction

Baseline theorem

For any function $b(s)$ that does **not** depend on a :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t)) \right]$$

The estimate remains **unbiased**.

Why? $\mathbb{E}_{\pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot b(s)] = 0$ because:

$$\sum_a \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s) = \nabla_{\theta} \sum_a \pi_{\theta}(a|s) = \nabla_{\theta} 1 = 0$$

Proof: Baseline has Zero Expectation

$$\begin{aligned}\mathbb{E}_\pi\left[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)\right] &= \int \pi_\theta(a|s) \nabla_\theta \log \pi_\theta(a|s) \cdot b(s) da \\ &= \int \nabla_\theta \pi_\theta(a|s) \cdot b(s) da \\ &= b(s) \nabla_\theta \underbrace{\int \pi_\theta(a|s) da}_{=1} = 0\end{aligned}$$

A baseline subtracts a constant from the reward signal without changing the gradient direction.

Common choice: $b(s_t) = V^\pi(s_t)$, giving the **advantage** $G_t - V^\pi(s_t) \approx A^\pi(s_t, a_t)$.

Policy Gradient with Advantage

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot A^{\pi}(s_t, a_t) \right]$$

Advantage Actor-Critic (A2C)

- **Actor:** policy π_{θ} — what to do
- **Critic:** value estimator $\hat{V}_{\phi}(s)$ — how good is the current state
- Advantage: $\hat{A}(s_t, a_t) = r_t + \gamma \hat{V}_{\phi}(s_{t+1}) - \hat{V}_{\phi}(s_t)$

Problem in LLMs

Learning a reliable critic $\hat{V}_{\phi}$ for natural-language states is *hard*.
GRPO (Lecture 3) eliminates the critic entirely.

Monte Carlo vs. Bootstrapping

Monte Carlo

$$G_t = \sum_{k \geq t} \gamma^{k-t} r_k$$

- Unbiased
- High variance
- Requires complete episodes

Temporal Difference (TD)

$$G_t \approx r_t + \gamma \hat{V}(s_{t+1})$$

- Biased (uses estimate)
- Lower variance
- Works online

GAE (Generalised Advantage Estimation) interpolates via $\lambda \in [0, 1]$:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Credit Assignment Noise

Problem

Which actions caused the high return? In a long trajectory, early and late rewards are entangled.

Causal structure: reward at t should only influence updates for actions $a_{t' \leq t}$:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \underbrace{\sum_{k \geq t} \gamma^{k-t} r_k}_{G_t} \right]$$

REINFORCE already captures this via G_t (future rewards only); the baseline sharpens it further.

Why this rules out a replay buffer

A replay buffer stores transitions (s_t, a_t, r_t, s_{t+1}) collected under an *old* policy $\pi_{\theta_{\text{old}}}$.

When we replay them under $\pi_{\theta} \neq \pi_{\theta_{\text{old}}}$, the credit assignment is **doubly broken**:

- G_t was computed under $\pi_{\theta_{\text{old}}}$ — the return reflects the old policy's downstream behaviour, not what π_{θ} would have done after a_t .
- The score $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ now pushes θ based on a reward signal that was never causally linked to π_{θ} .

Upshot: policy gradient must be on-policy, or use importance weighting — the cost GRPO avoids by treating each prompt as a single-step reward.

Towards Stable Policy Updates: PPO

The problem with large gradient steps

A big update to θ can move π_θ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

Policy collapse: a few bad steps can destroy a well-trained policy irreversibly.

Towards Stable Policy Updates: PPO

The problem with large gradient steps

A big update to θ can move π_θ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

Policy collapse: a few bad steps can destroy a well-trained policy irreversibly.

TRPO (2015) formalises this as a KL-constrained optimisation problem.

Towards Stable Policy Updates: PPO

The problem with large gradient steps

A big update to θ can move π_θ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

Policy collapse: a few bad steps can destroy a well-trained policy irreversibly.

TRPO (2015) formalises this as a KL-constrained optimisation problem.

Proximal Policy Optimisation (PPO) (Schulman et al., 2017)

Replace the constraint with a **clipped surrogate objective**:

$$J^{\text{PPO}}(\theta) = \mathbb{E}_t[\min(\rho_t A_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon) A_t)]$$

where $\rho_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the **importance ratio** — how much the policy has moved.

The clip removes the incentive to push ρ_t outside $[1-\varepsilon, 1+\varepsilon]$, bounding each step.

Lecture 2 — Summary

Core take-aways

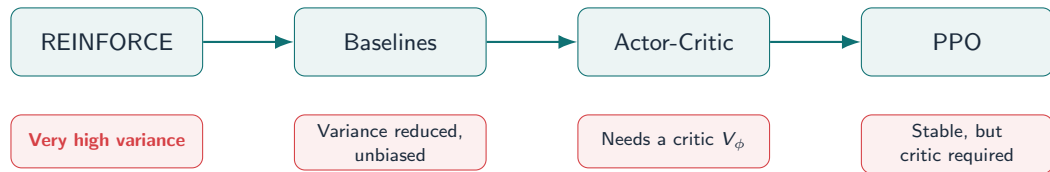
1. **Log-derivative trick:** $\nabla_{\theta} J = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta} \cdot G]$
2. **REINFORCE:** unbiased, but high-variance gradient estimator
3. **Baselines** reduce variance without introducing bias
4. **Advantage** $A^{\pi}(s, a)$ is the centred signal of choice
5. **Actor-Critic:** use a learned V^{π} as baseline — but critic training is hard

Next: eliminate the critic entirely using group comparisons.

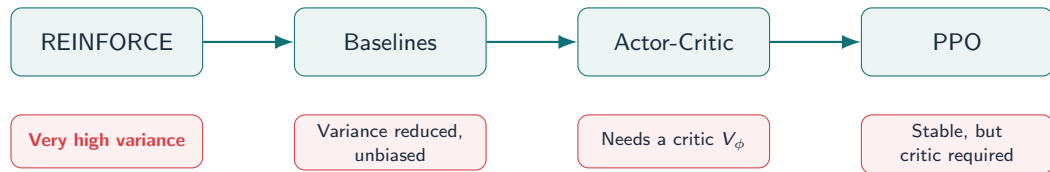
Lecture 3

Group Relative Policy Optimization (GRPO)

Recap



Recap



The unresolved question

PPO works — but it requires a **critic network** V_ϕ (same size as the policy) trained via TD.

Today: can we do better?

PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

1. Critic update (TD)

Train V_ϕ to predict $V^\pi(s)$ by minimising the **TD error**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Bootstraps: the target uses the current V_ϕ estimate of the next state.

PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

1. Critic update (TD)

Train V_ϕ to predict $V^\pi(s)$ by minimising the **TD error**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Bootstraps: the target uses the current V_ϕ estimate of the next state.

2. Actor update (policy gradient)

Update π_θ using the clipped surrogate:

$$J^{\text{PPO}} = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right]$$

$\rho_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the importance ratio and $\hat{A}_t$ comes from the critic.

PPO Has Two Distinct Updates

PPO is an **Actor-Critic** method. At each step it performs two separate optimisation updates:

1. Critic update (TD)

Train V_ϕ to predict $V^\pi(s)$ by minimising the **TD error**:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Bootstraps: the target uses the current V_ϕ estimate of the next state.

2. Actor update (policy gradient)

Update π_θ using the clipped surrogate:

$$J^{\text{PPO}} = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right]$$

$\rho_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ is the importance ratio and $\hat{A}_t$ comes from the critic.

TD solves: *how good is this state?* **Policy gradient** solves: *which direction should θ move?*

Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid

Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

$$\lambda = 0$$

One-step TD

$$\hat{A}_t = \delta_t$$

Low variance

High bias

Trusts the critic immediately after one step.

Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

$$\lambda = 0$$

One-step TD

$$\hat{A}_t = \delta_t$$

Low variance

High bias

Trusts the critic immediately after one step.

$$\lambda = 1$$

Monte Carlo

$$\hat{A}_t = G_t - V_\phi(s_t)$$

Unbiased

High variance

Uses all real rewards; never relies on the critic for future steps.

Advantage Estimate

The one-step estimate $\hat{A}(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is valid — but it **relies entirely on** $V_\phi(s_{t+1})$ being accurate. If the critic is wrong, the whole advantage estimate is wrong.

$\lambda = 0$

One-step TD

$$\hat{A}_t = \delta_t$$

Low variance

High bias

Trusts the critic immediately after one step.

$\lambda = 1$

Monte Carlo

$$\hat{A}_t = G_t - V_\phi(s_t)$$

Unbiased

High variance

Uses all real rewards; never relies on the critic for future steps.

$\lambda \approx 0.95$

GAE (PPO default)

$$\hat{A}_t = \sum_k (\gamma \lambda)^k \delta_{t+k}$$

Low bias

Moderate variance

Observes ≈ 20 real steps before bootstrapping. Critic errors get diluted.

$$\hat{A}_t = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

The Critic is the Bottleneck

Why critic training is hard for LLMs:

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_{\phi}(s)$ must assign a scalar value to an arbitrary token sequence

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_{\phi}(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_{\phi}(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_{\phi}(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

The core problem

The critic adds a **full copy** of the model in memory, doubles the training complexity, and is the most common source of instability in PPO.

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

The core problem

The critic adds a **full copy** of the model in memory, doubles the training complexity, and is the most common source of instability in PPO.

The key question

Can we estimate a useful baseline **without training V_ϕ at all?**

The Critic is the Bottleneck

Why critic training is hard for LLMs:

- $V_\phi(s)$ must assign a scalar value to an arbitrary token sequence
- Rewards are sparse
- The critic is chasing a *moving target*
- Training instability compounds

The core problem

The critic adds a **full copy** of the model in memory, doubles the training complexity, and is the most common source of instability in PPO.

The key question

Can we estimate a useful baseline **without training V_ϕ at all?**

The answer

Yes – replace the critic with the empirical group mean. This is GRPO (Shao et al 2024).

PPO vs. GRPO: What Changes

	PPO	GRPO
<i>Models loaded in memory:</i>		
Policy π_θ (being trained)	✓	✓
Reference policy π_{old} (frozen SFT)	✓	✓
Critic V_ϕ (value network, same size as policy)	✓	×
Reward model r_ϕ	✓	✓ / × [†]
Total models in memory	4	3 (or 2[†])
Critic / value function training required	Yes	No
Advantage estimation	TD error via V_ϕ	Group mean $\bar{r}$
Approx. GPU memory (70B, bf16)	~1.6 TB	~1.2 TB

[†] When rewards are *verifiable* (maths, code), the reward model is replaced by a rule-based function — no separate model needed. DeepSeek-R1 uses this approach.

Real-World Deployments

GRPO is now the standard RL algorithm for frontier reasoning models:

DeepSeekMath (2024)

Task: mathematical reasoning

Result: state-of-the-art on MATH benchmark using GRPO, outperforming PPO-trained models of the same size

Key finding: group-relative advantages work especially well when rewards are *verifiable* (correct/incorrect) rather than learned

Shao et al., arXiv:2402.03300

DeepSeek-R1 (2025)

Task: general reasoning (math, code, science)

Result: competitive with OpenAI o1 at a fraction of the training cost

Architecture: GRPO at every RL stage; no critic; rule-based verifiable rewards — **only 2 models in memory**

DeepSeek-AI, arXiv:2501.12948

Models trained with GRPO or direct variants:

- **Kimi k1.5** (Moonshot AI, 2025):
long-context RL with group-relative rewards;
arXiv:2501.12599
- **Qwen2.5-Math** (Alibaba, 2024): GRPO for
mathematical chain-of-thought reasoning;
arXiv:2409.12122
- **Open-source reproductions:**
Light-R1, SimpleRL, and others confirming
results with <8 GPU training runs

Paradigm shift

Before GRPO: RL for LLMs required large research infrastructure.

After GRPO: a single research group can run competitive RL training on a cluster of 8 GPUs.

GRPO democratised LLM RL.

Key Idea: Relative Comparisons

Insight

We don't need the *absolute* quality of a response.

We need to know: **is this response better or worse than typical?**

Key Idea: Relative Comparisons

Insight

We don't need the *absolute* quality of a response.

We need to know: **is this response better or worse than typical?**

Human analogy: a teacher grades an essay not by a fixed scale, but relative to other essays submitted. **GRPO:** for each prompt q , sample K responses and rank them *against each other*.

Key Idea: Relative Comparisons

Insight

We don't need the *absolute* quality of a response.

We need to know: **is this response better or worse than typical?**

Human analogy: a teacher grades an essay not by a fixed scale, but relative to other essays submitted. **GRPO:** for each prompt q , sample K responses and rank them *against each other*.

- No separate critic required
- Baseline is computed from the group itself
- Stable across training

Step 1: Sample a Group of Responses

Given prompt q (a context / instruction):

$$a_1, a_2, \dots, a_K \stackrel{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- K is the **group size** (e.g. $K = 8$ or $K = 16$)

Step 1: Sample a Group of Responses

Given prompt q (a context / instruction):

$$a_1, a_2, \dots, a_K \stackrel{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- K is the **group size** (e.g. $K = 8$ or $K = 16$)
- Responses are sampled from the **old policy** π_{old} (policy before current update)

Step 1: Sample a Group of Responses

Given prompt q (a context / instruction):

$$a_1, a_2, \dots, a_K \stackrel{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- K is the **group size** (e.g. $K = 8$ or $K = 16$)
- Responses are sampled from the **old policy** π_{old} (policy before current update)
- Each a_i receives a reward $r_i = R(q, a_i)$ from a reward model

Step 1: Sample a Group of Responses

Given prompt q (a context / instruction):

$$a_1, a_2, \dots, a_K \stackrel{\text{iid}}{\sim} \pi_{\text{old}}(\cdot \mid q)$$

- K is the **group size** (e.g. $K = 8$ or $K = 16$)
- Responses are sampled from the **old policy** π_{old} (policy before current update)
- Each a_i receives a reward $r_i = R(q, a_i)$ from a reward model

In LLM practice

q = a user prompt; a_i = a complete generated response; R = reward model score (helpfulness, safety, etc.)

Step 2: Group Mean Baseline

Compute the empirical mean reward:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i$$

This is the **group-relative baseline** — the average quality of responses to this prompt.

Step 2: Group Mean Baseline

Compute the empirical mean reward:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i$$

This is the **group-relative baseline** — the average quality of responses to this prompt.

Why is this a valid baseline?

As $K \rightarrow \infty$:

$$\bar{r} \xrightarrow{\text{a.s.}} \mathbb{E}_{a \sim \pi_{\text{old}}} [R(q, a)] = V^{\pi_{\text{old}}}(q)$$

With finite K , it is a *noisy but unbiased* estimate of the value.

Step 3: Normalised Advantages

GRPO advantage estimate

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$$

where $\sigma_r = \sqrt{\frac{1}{K} \sum_i (r_i - \bar{r})^2}$ and $\epsilon > 0$ for numerical stability.

Step 3: Normalised Advantages

GRPO advantage estimate

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$$

where $\sigma_r = \sqrt{\frac{1}{K} \sum_i (r_i - \bar{r})^2}$ and $\epsilon > 0$ for numerical stability.

- **Centering**: subtracts the baseline (variance reduction)
- **Scaling**: divides by the std. dev. (stabilises gradient magnitudes)
- **Key property**: $\sum_{i=1}^K \hat{A}_i = 0$ by construction

Key Property: Zero-Sum Advantages

$$\sum_{i=1}^K \hat{A}_i = \sum_{i=1}^K \frac{r_i - \bar{r}}{\sigma_r} = \frac{1}{\sigma_r} \underbrace{\sum_{i=1}^K (r_i - \bar{r})}_{=0} = 0$$

Interpretation:

- Increasing probability of good responses *must* decrease probability of bad ones
- The update is a **zero-sum redistribution** of probability mass within the group
- This is a form of *contrastive learning* applied to RL

GRPO turns a reward signal into a relative ranking signal.

The GRPO Objective

GRPO surrogate loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{K} \sum_{i=1}^K \min\left(\rho_i(\theta) \hat{A}_i, \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i\right)$$

where the **importance ratio** is $\rho_i(\theta) = \frac{\pi_{\theta}(\mathbf{a}_i \mid \mathbf{q})}{\pi_{\text{old}}(\mathbf{a}_i \mid \mathbf{q})}$.

The GRPO Objective

GRPO surrogate loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{K} \sum_{i=1}^K \min\left(\rho_i(\theta) \hat{A}_i, \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i\right)$$

where the **importance ratio** is $\rho_i(\theta) = \frac{\pi_{\theta}(\mathbf{a}_i \mid \mathbf{q})}{\pi_{\text{old}}(\mathbf{a}_i \mid \mathbf{q})}$.

- The clip prevents large policy updates (PPO-style)

The GRPO Objective

GRPO surrogate loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{K} \sum_{i=1}^K \min\left(\rho_i(\theta) \hat{A}_i, \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i\right)$$

where the **importance ratio** is $\rho_i(\theta) = \frac{\pi_{\theta}(\mathbf{a}_i \mid \mathbf{q})}{\pi_{\text{old}}(\mathbf{a}_i \mid \mathbf{q})}$.

- The `clip` prevents large policy updates (PPO-style)
- $\hat{A}_i$ is the normalised group advantage from the previous slide

The GRPO Objective

GRPO surrogate loss

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{K} \sum_{i=1}^K \min\left(\rho_i(\theta) \hat{A}_i, \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_i\right)$$

where the **importance ratio** is $\rho_i(\theta) = \frac{\pi_{\theta}(\mathbf{a}_i \mid \mathbf{q})}{\pi_{\text{old}}(\mathbf{a}_i \mid \mathbf{q})}$.

- The `clip` prevents large policy updates (PPO-style)
- $\hat{A}_i$ is the normalised group advantage from the previous slide
- Optimise over a **mini-batch** of prompts $\mathbf{q}$

Responses with $\hat{A}_i > 0$

(above-average reward)

- Importance ratio $\rho_i > 1$ increases the objective
- Policy is pushed to **make them more likely**

Interpreting the GRPO Update

Responses with $\hat{A}_i > 0$

(above-average reward)

- Importance ratio $\rho_i > 1$ increases the objective
- Policy is pushed to **make them more likely**

Responses with $\hat{A}_i < 0$

(below-average reward)

- Importance ratio $\rho_i > 1$ *decreases* the objective
- Policy is pushed to **make them less likely**

Code: Bandit GRPO — Setup

Simplest possible GRPO: a K -armed bandit (no state, just actions and rewards).

This is the exact structure of GRPO applied to a single prompt.

```
1 import torch
2 import torch.nn as nn
3
4 # Policy over N candidate responses
5 class BanditPolicy(nn.Module):
6     def __init__(self, n_actions=10):
7         super().__init__()
8         # Learnable logits = log-unnormalised probs
9         self.logits = nn.Parameter(
10             torch.zeros(n_actions)
11         )
12
13     def distribution(self):
14         return torch.distributions.Categorical(
15             logits=self.logits
16         )
17
18 def proxy_reward(actions):
19     """Stand-in for a reward model.
20     Higher action index -> better response."""
21     signal = actions.float() * 0.4
22     noise = torch.randn_like(signal) * 0.3
23     return signal + noise
```

Why a bandit?

- No state transitions to worry about
- Each “action” = one generated response
- Reward = score from a reward model
- Captures the *entire* GRPO mechanism

Mapping to LLMs:

$n_actions \approx \text{vocabulary}^{\text{seq_len}}$
(exponentially large); in practice we sample rather than enumerate.

Code: Bandit GRPO — Training Loop

```
1 policy      = BanditPolicy(n_actions=10)
2 optimizer   = torch.optim.Adam(
3     policy.parameters(), lr=0.05
4 )
5
6 for step in range(500):
7     K      = 8 # group size
8     dist   = policy.distribution()
9
10    # 1. Sample group
11    actions = dist.sample((K,)) # [K]
12    rewards = proxy_reward(actions) # [K]
13
14    # 2. GRPO advantages (centred + scaled)
15    A_hat   = (rewards - rewards.mean()) \
16        / (rewards.std() + 1e-8) # [K]
17
18    # 3. Policy gradient (importance ratio = 1
19    #     because we re-sample each step)
20    log_probs = dist.log_prob(actions) # [K]
21    loss      = -(log_probs * A_hat.detach()).mean()
22
23    optimizer.zero_grad()
24    loss.backward()
25    optimizer.step()
```

What to watch:

- `torch.softmax(policy.logits, 0)` — watch probability mass shift toward high-index actions
- `A_hat.sum() = 0` always (by construction)
- Remove normalisation (`std`) and observe gradient instability
- Try $K = 1$ — no update (single sample, $\hat{A}_1 = 0$)

Exercise

Replace `proxy_reward` with a function that scores responses by length. Observe reward hacking: policy collapses to the longest action.

Fix: add KL penalty against initial logits.

Why We Need Regularisation

Without constraints

Maximising only the reward encourages the policy to drift arbitrarily far from π_{old} :

- Importance ratios ρ_i become extreme $\rightarrow$ high-variance updates
- Policy can collapse to degenerate responses (reward hacking)
- Distribution shift invalidates the importance-weighted estimate

Why We Need Regularisation

Without constraints

Maximising only the reward encourages the policy to drift arbitrarily far from π_{old} :

- Importance ratios ρ_i become extreme $\rightarrow$ high-variance updates
- Policy can collapse to degenerate responses (reward hacking)
- Distribution shift invalidates the importance-weighted estimate

Solution: add a **KL divergence penalty** to constrain how far the policy moves.

Regularised GRPO

$$\max_{\theta} \mathbb{E}_q \left[\frac{1}{K} \sum_{i=1}^K \rho_i(\theta) \hat{A}_i \right] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{old}})$$

- $\beta > 0$: regularisation strength (hyperparameter)
- $\text{KL}(\pi_{\theta} \parallel \pi_{\text{old}}) = \mathbb{E}_{\theta} \left[\log \frac{\pi_{\theta}(a|q)}{\pi_{\text{old}}(a|q)} \right]$
- Larger β : more conservative updates; smaller β : more aggressive learning

Lagrangian Derivation of the Optimal Policy

Constrained form:

$$\max_{\pi} \mathbb{E}_q[\mathbb{E}_{a \sim \pi_{\text{old}}}[\rho(\theta) A]] \quad \text{s.t.} \quad \text{KL}(\pi \parallel \pi_{\text{old}}) \leq \delta$$

Lagrangian Derivation of the Optimal Policy

Constrained form:

$$\max_{\pi} \mathbb{E}_q[\mathbb{E}_{a \sim \pi_{\text{old}}}[\rho(\theta) A]] \quad \text{s.t.} \quad \text{KL}(\pi \| \pi_{\text{old}}) \leq \delta$$

Lagrangian:

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{a \sim \pi_{\text{old}}} \left[\frac{\pi(a)}{\pi_{\text{old}}(a)} A(a) \right] - \lambda \text{KL}(\pi \| \pi_{\text{old}})$$

Lagrangian Derivation of the Optimal Policy

Constrained form:

$$\max_{\pi} \mathbb{E}_q[\mathbb{E}_{a \sim \pi_{\text{old}}}[\rho(\theta) A]] \quad \text{s.t.} \quad \text{KL}(\pi \| \pi_{\text{old}}) \leq \delta$$

Lagrangian:

$$\mathcal{L}(\pi, \lambda) = \mathbb{E}_{a \sim \pi_{\text{old}}} \left[\frac{\pi(a)}{\pi_{\text{old}}(a)} A(a) \right] - \lambda \text{KL}(\pi \| \pi_{\text{old}})$$

Setting $\frac{\delta \mathcal{L}}{\delta \pi(a)} = 0$ and solving:

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

Exponential Tilting

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

This is **exponential tilting** (also called Gibbs/Boltzmann distribution):

Exponential Tilting

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

This is **exponential tilting** (also called Gibbs/Boltzmann distribution):

- Actions with high advantage receive *exponentially* higher probability
- λ controls the sharpness (temperature)
- As $\lambda \rightarrow 0$: greedy (argmax); as $\lambda \rightarrow \infty$: uniform ($= \pi_{\text{old}}$)

Exponential Tilting

$$\pi^*(a \mid q) \propto \pi_{\text{old}}(a \mid q) \cdot \exp\left(\frac{A(a, q)}{\lambda}\right)$$

This is **exponential tilting** (also called Gibbs/Boltzmann distribution):

- Actions with high advantage receive *exponentially* higher probability
- λ controls the sharpness (temperature)
- As $\lambda \rightarrow 0$: greedy (argmax); as $\lambda \rightarrow \infty$: uniform ($= \pi_{\text{old}}$)

Information-geometric view

The KL constraint defines a ball in distribution space.

The optimal policy is the *projection* of the reward-weighted distribution onto this ball.

GRPO vs. Vanilla Policy Gradient

	Policy Gradient	GRPO
Baseline	Learned critic $\hat{V}_\phi(s)$	Empirical group mean $\bar{r}$
Extra network	Yes (critic)	No
Advantage source	TD error	Within-group ranking
Bias	Low (w/ good critic)	Finite- K bias
Variance	High w/o critic	Low via centering
Stability	Critic divergence risk	KL-constrained

GRPO for RLHF

Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

1. **Ranking-based:** updates depend only on *relative* ordering of responses
⇒ humans naturally provide rankings, not absolute scores

Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

1. **Ranking-based:** updates depend only on *relative* ordering of responses
⇒ humans naturally provide rankings, not absolute scores
2. **Critic-free:** no separate value network to train or stabilise
⇒ human labels are noisy; a critic would fit noise

Why GRPO is the Right Foundation for Human Feedback

GRPO has three properties that make it ideal for learning from humans:

1. **Ranking-based:** updates depend only on *relative* ordering of responses
⇒ humans naturally provide rankings, not absolute scores
2. **Critic-free:** no separate value network to train or stabilise
⇒ human labels are noisy; a critic would fit noise
3. **Stable under noisy signals:** normalised advantages $\hat{A}_i = (r_i - \bar{r})/\sigma_r$
⇒ annotation noise is absorbed by the group standard deviation σ_r

Human Feedback as Ranking Data

What annotators provide:

- Pairwise comparisons: “ $A \succ B$ ”
- Rankings of K outputs:
 $a_{\sigma(1)} \succ \cdots \succ a_{\sigma(K)}$
- Preference labels with optional comments

Human Feedback as Ranking Data

What annotators provide:

- Pairwise comparisons: “A $\succ$ B”
- Rankings of K outputs:
 $a_{\sigma(1)} \succ \cdots \succ a_{\sigma(K)}$
- Preference labels with optional comments

What annotators don't provide:

- Absolute scores
- Probability values
- Why they prefer one over another

Human Feedback as Ranking Data

What annotators provide:

- Pairwise comparisons: “A $\succ$ B”
- Rankings of K outputs:
 $a_{\sigma(1)} \succ \dots \succ a_{\sigma(K)}$
- Preference labels with optional comments

What annotators don't provide:

- Absolute scores
- Probability values
- Why they prefer one over another

Key observation

Human judgements are **inherently relative**.

“Response A is better than Response B”
is easier and more reliable than
“Response A scores 7.3 / 10”.

This is exactly what GRPO exploits.

Mapping RLHF Concepts onto GRPO

RLHF concept	GRPO counterpart
Human rankings of K outputs	Group samples $a_1, \dots, a_K \sim \pi_{\text{old}}$
Preferred response	High reward $r_i \Rightarrow \hat{A}_i > 0$
Dispreferred response	Low reward $r_i \Rightarrow \hat{A}_i < 0$
Implicit quality baseline	Group mean $\bar{r} = \frac{1}{K} \sum r_i$
Reward model score	Scalar $r_i = r_\phi(q, a_i)$
KL constraint (stay safe)	$\beta \text{KL}(\pi_\theta \ \pi_{\text{ref}})$

The Bridge Statement

“RLHF is just GRPO where the reward signal comes from humans instead of the environment.”

Environment reward (classic RL)

$r_i = R(s, a_i)$ — defined by the MDP

Deterministic or stochastic

Objective: maximise $J(\theta)$

Human reward (RLHF)

$r_i = r_\phi(q, a_i)$ — predicted by reward model

Noisy, biased, context-dependent

Objective: align with human intent

Why This Matters

The GRPO/RLHF correspondence has three important consequences:

Why This Matters

The GRPO/RLHF correspondence has three important consequences:

1. **No true reward function required**

We replace the intractable “true reward” with human comparative judgements

Why This Matters

The GRPO/RLHF correspondence has three important consequences:

1. **No true reward function required**

We replace the intractable “true reward” with human comparative judgements

2. **Reduced dependence on value function estimation**

The group mean $\bar{r}$ is a computable proxy — no learned critic needed

This is critical when the state space is natural language

Why This Matters

The GRPO/RLHF correspondence has three important consequences:

1. **No true reward function required**

We replace the intractable “true reward” with human comparative judgements

2. **Reduced dependence on value function estimation**

The group mean $\bar{r}$ is a computable proxy — no learned critic needed

This is critical when the state space is natural language

3. **Relative structure arises naturally**

Human preference is ordinal; GRPO training signal is ordinal

No mismatch between data structure and algorithm

**Practical Application:
Training LLMs with Verifiable Rewards**

- **No Reward Model (RM) needed:** For tasks with a ground truth, we use rule-based code to verify answers.
- **Massive Throughput:** We generate multiple answers in parallel to build our reference group baseline.

DeepSeek-R1 Insight: Verifiable Rewards

Why does this work so well for Math and Code? We replace noisy human-like scoring with **objective rules**.

Accuracy Reward

Does the answer match the target?

- Correct: +1.0
- Incorrect: 0.0

Format Reward

Does it follow the structure?

- Used `<thought>`?
- Used `\boxed{}`?

This removes "Reward Model bias" because the signal is factual truth.

A Concrete Example: The "Tournament"

Prompt: "What is the square root of 144? Answer in a box."

ID	Model Content	Correct?	Reward	Advantage
a_1	"Answer is 12 ."	Yes	1.0	+0.98
a_2	" $12 \times 12 = 144$. 12 "	Yes	1.0	+0.98
a_3	"It is 14. 14 "	No	0.0	-1.02
a_4	"I think it is 72 "	No	0.0	-1.02

- **Group Baseline:** Mean $\bar{r} = 0.5$.
- **Learning Signal:** Responses a_1 and a_2 are reinforced. a_3 and a_4 are suppressed.
- **Outcome:** The model discovers that valid math and formatting lead to "winning."

Lecture 3 — Summary

Core take-aways

1. **GRPO** replaces the expensive critic with a group-mean baseline.
2. **Verifiable Rewards** (Math/Code) make GRPO extremely stable.
3. Centering rewards ($\hat{A}_i = \frac{r_i - \bar{r}}{\sigma}$) provides the variance reduction.
4. **KL regularisation** is critical to prevent the model from "cheating."
5. **DeepSeek-R1** proved that this is the path to competitive reasoning.
6. RLHF = GRPO logic, but with human rankings as the reward source.

Next: When does this work in theory? (Lecture 4 — Convergence)

Lecture 4

Theory: Variance, Bias, and Convergence

Why is RL Noisy?

The policy gradient estimator:

$$\hat{g} = \frac{1}{N} \sum_n \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n) \cdot G_t^n.$$

Why is RL Noisy?

The policy gradient estimator:

$$\hat{g} = \frac{1}{N} \sum_n \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^n | s_t^n) \cdot G_t^n.$$

Four sources of variance:

1. **Policy stochasticity** — different actions each episode
2. **Environmental noise** — stochastic transitions $P(s' | s, a)$
3. **Long horizons** — returns compound uncertainty over T steps
4. **Score function** — $\nabla_{\theta} \log \pi_{\theta}$ can have large norm

Variance Decomposition

Per-step contribution: $X_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$

Variance Decomposition

Per-step contribution: $X_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$

Per-trajectory contribution:

$$X = \sum_{t=0}^T X_t = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$$

Variance Decomposition

Per-step contribution: $X_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$

Per-trajectory contribution:

$$X = \sum_{t=0}^T X_t = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$$

With i.i.d. trajectories, $\hat{g} = \frac{1}{N} \sum_n X^{(n)}$, hence

$$\text{Var}(\hat{g}) = \frac{1}{N} \text{Var}(X) = \frac{1}{N} \left[\mathbb{E}[X^2] - (\mathbb{E}[X])^2 \right].$$

Variance Decomposition

Per-step contribution: $X_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$.

Per-trajectory contribution:

$$X = \sum_{t=0}^T X_t = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$$

With i.i.d. trajectories, $\hat{g} = \frac{1}{N} \sum_n X^{(n)}$, hence

$$\text{Var}(\hat{g}) = \frac{1}{N} \text{Var}(X) = \frac{1}{N} [\mathbb{E}[X^2] - (\mathbb{E}[X])^2].$$

What each term is:

- $\mathbb{E}[X^2]$ — grows with the horizon T ; returns accumulate noise across steps
- $(\mathbb{E}[X])^2 = \|\nabla_{\theta} J(\theta)\|^2$ — the *signal* we want
- Signal-to-noise ratio (SNR) degrades as $T \rightarrow \infty$

Variance Decomposition

Per-step contribution: $X_t = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t$.

Per-trajectory contribution:

$$X = \sum_{t=0}^T X_t = \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot G_t.$$

With i.i.d. trajectories, $\hat{g} = \frac{1}{N} \sum_n X^{(n)}$, hence

$$\text{Var}(\hat{g}) = \frac{1}{N} \text{Var}(X) = \frac{1}{N} [\mathbb{E}[X^2] - (\mathbb{E}[X])^2].$$

What each term is:

- $\mathbb{E}[X^2]$ — grows with the horizon T ; returns accumulate noise across steps
- $(\mathbb{E}[X])^2 = \|\nabla_{\theta} J(\theta)\|^2$ — the *signal* we want
- Signal-to-noise ratio (SNR) degrades as $T \rightarrow \infty$

$$\text{Var}(G_t) \sim O(T) \implies \text{Var}(\hat{g}) \sim O(T/N).$$

Deriving the Variance of the Return

Return from time 0:

$$G_0 = \sum_{t=0}^T \gamma^t r_t.$$

Deriving the Variance of the Return

Return from time 0:

$$G_0 = \sum_{t=0}^T \gamma^t r_t.$$

Assumption. Per-step rewards r_t are i.i.d. with variance σ_r^2 .

Deriving the Variance of the Return

Return from time 0:

$$G_0 = \sum_{t=0}^T \gamma^t r_t.$$

Assumption. Per-step rewards r_t are i.i.d. with variance σ_r^2 .

Step 1 — variance of a sum of independent terms.

$$\text{Var}(G_0) = \text{Var}\left(\sum_{t=0}^T \gamma^t r_t\right) = \sum_{t=0}^T \gamma^{2t} \text{Var}(r_t) = \sigma_r^2 \sum_{t=0}^T \gamma^{2t}.$$

Deriving the Variance of the Return

Return from time 0:

$$G_0 = \sum_{t=0}^T \gamma^t r_t.$$

Assumption. Per-step rewards r_t are i.i.d. with variance σ_r^2 .

Step 1 — variance of a sum of independent terms.

$$\text{Var}(G_0) = \text{Var}\left(\sum_{t=0}^T \gamma^t r_t\right) = \sum_{t=0}^T \gamma^{2t} \text{Var}(r_t) = \sigma_r^2 \sum_{t=0}^T \gamma^{2t}.$$

Step 2 — finite geometric series.

$$\sum_{t=0}^T \gamma^{2t} = \frac{1 - \gamma^{2(T+1)}}{1 - \gamma^2}.$$

Deriving the Variance of the Return

Return from time 0:

$$G_0 = \sum_{t=0}^T \gamma^t r_t.$$

Assumption. Per-step rewards r_t are i.i.d. with variance σ_r^2 .

Step 1 — variance of a sum of independent terms.

$$\text{Var}(G_0) = \text{Var}\left(\sum_{t=0}^T \gamma^t r_t\right) = \sum_{t=0}^T \gamma^{2t} \text{Var}(r_t) = \sigma_r^2 \sum_{t=0}^T \gamma^{2t}.$$

Step 2 — finite geometric series.

$$\sum_{t=0}^T \gamma^{2t} = \frac{1 - \gamma^{2(T+1)}}{1 - \gamma^2}.$$

Step 3 — combine:

$$\text{Var}(G_0) = \sigma_r^2 \cdot \frac{1 - \gamma^{2(T+1)}}{1 - \gamma^2}.$$

Horizon Scaling Problem

Discounted, long horizon ($\gamma < 1$, $T \rightarrow \infty$):

$$\text{Var}(G_0) \xrightarrow{T \rightarrow \infty} \frac{\sigma_r^2}{1 - \gamma^2}.$$

Variance *saturates*. For $\gamma = 0.99$: $1 - \gamma^2 \approx 0.02$, giving $\text{Var}(G_0) \approx 50 \sigma_r^2$.

Horizon Scaling Problem

Discounted, long horizon ($\gamma < 1$, $T \rightarrow \infty$):

$$\text{Var}(G_0) \xrightarrow{T \rightarrow \infty} \frac{\sigma_r^2}{1 - \gamma^2}.$$

Variance *saturates*. For $\gamma = 0.99$: $1 - \gamma^2 \approx 0.02$, giving $\text{Var}(G_0) \approx 50 \sigma_r^2$.

Undiscounted ($\gamma = 1$):

$$\text{Var}(G_0) = (T + 1) \sigma_r^2 \sim O(T).$$

Variance grows *linearly* with horizon.

Horizon Scaling Problem

Discounted, long horizon ($\gamma < 1$, $T \rightarrow \infty$):

$$\text{Var}(G_0) \xrightarrow{T \rightarrow \infty} \frac{\sigma_r^2}{1 - \gamma^2}.$$

Variance *saturates*. For $\gamma = 0.99$: $1 - \gamma^2 \approx 0.02$, giving $\text{Var}(G_0) \approx 50 \sigma_r^2$.

Undiscounted ($\gamma = 1$):

$$\text{Var}(G_0) = (T + 1) \sigma_r^2 \sim O(T).$$

Variance grows *linearly* with horizon.

LLM consequence

In LLM RL we typically use $\gamma = 1$: the task is the whole response, we don't want to discount the final answer. Responses of 500–2000 tokens. Sparse end-of-sequence rewards make the problem strictly worse.

Estimating the Covariance of $\hat{g}$

Definition. For any unbiased estimator with $\mathbb{E}[\hat{g}] = \nabla_{\theta} J(\theta)$:

$$\Sigma_{\hat{g}} = \mathbb{E}[(\hat{g} - \nabla_{\theta} J)(\hat{g} - \nabla_{\theta} J)^{\top}] \in \mathbb{R}^{d \times d}.$$

Estimating the Covariance of $\hat{g}$

Definition. For any unbiased estimator with $\mathbb{E}[\hat{g}] = \nabla_{\theta} J(\theta)$:

$$\Sigma_{\hat{g}} = \mathbb{E}[(\hat{g} - \nabla_{\theta} J)(\hat{g} - \nabla_{\theta} J)^{\top}] \in \mathbb{R}^{d \times d}.$$

i.i.d. trajectories. Write $\hat{g} = \frac{1}{N} \sum_{n=1}^N X_n$, with

$$X_n = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^{(n)} | s_t^{(n)}) \cdot G_t^{(n)}.$$

Then $\Sigma_{\hat{g}} = \frac{1}{N} \Sigma_X$, estimated from the sample by

$$\hat{\Sigma}_{\hat{g}} = \frac{1}{N(N-1)} \sum_{n=1}^N (X_n - \bar{X})(X_n - \bar{X})^{\top}.$$

Estimating the Covariance of $\hat{g}$

Definition. For any unbiased estimator with $\mathbb{E}[\hat{g}] = \nabla_{\theta} J(\theta)$:

$$\Sigma_{\hat{g}} = \mathbb{E}[(\hat{g} - \nabla_{\theta} J)(\hat{g} - \nabla_{\theta} J)^{\top}] \in \mathbb{R}^{d \times d}.$$

i.i.d. trajectories. Write $\hat{g} = \frac{1}{N} \sum_{n=1}^N X_n$, with

$$X_n = \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t^{(n)} | s_t^{(n)}) \cdot G_t^{(n)}.$$

Then $\Sigma_{\hat{g}} = \frac{1}{N} \Sigma_X$, estimated from the sample by

$$\hat{\Sigma}_{\hat{g}} = \frac{1}{N(N-1)} \sum_{n=1}^N (X_n - \bar{X})(X_n - \bar{X})^{\top}.$$

For LLMs $d \sim 10^{10}$ — forming the full matrix is impossible. Usually the trace, diagonal, or top-k eigen values are reported.

Effective Sample Size

General definition.

$$N_{\text{eff}} = \frac{\text{Var}_{\text{iid}}(\hat{g})}{\text{Var}_{\text{actual}}(\hat{g})} \cdot N.$$

Calibrated so $N_{\text{eff}} = N$ exactly under i.i.d. sampling; correlation or unequal weighting can only reduce it.

Effective Sample Size

General definition.

$$N_{\text{eff}} = \frac{\text{Var}_{\text{iid}}(\hat{g})}{\text{Var}_{\text{actual}}(\hat{g})} \cdot N.$$

Calibrated so $N_{\text{eff}} = N$ exactly under i.i.d. sampling; correlation or unequal weighting can only reduce it.

Kish's formula (Kish, *Survey Sampling*, 1965; weights w_i):

$$N_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

Origin: the IS estimator has variance $\propto \sum w_i^2$; equate to the i.i.d. variance σ^2/N and solve.
Equal weights $\Rightarrow N_{\text{eff}} = N$ (check it).

Effective Sample Size

General definition.

$$N_{\text{eff}} = \frac{\text{Var}_{\text{iid}}(\hat{g})}{\text{Var}_{\text{actual}}(\hat{g})} \cdot N.$$

Calibrated so $N_{\text{eff}} = N$ exactly under i.i.d. sampling; correlation or unequal weighting can only reduce it.

Kish's formula (Kish, *Survey Sampling*, 1965; weights w_i):

$$N_{\text{eff}} = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}.$$

Origin: the IS estimator has variance $\propto \sum w_i^2$; equate to the i.i.d. variance σ^2/N and solve. Equal weights $\Rightarrow N_{\text{eff}} = N$ (check it).

Example. $N = 5$ with weights $(1, 1, 1, 1, 10)$: $(\sum w)^2 = 196$, $\sum w^2 = 104$, so $N_{\text{eff}} \approx 1.88$.

When ESS Drops in GRPO

ESS falls below N through *correlation*, *unequal weighting*, or both. Both appear in GRPO.

When ESS Drops in GRPO

ESS falls below N through *correlation*, *unequal weighting*, or both. Both appear in GRPO.

Case 1: GRPO group of K correlated samples

$\sum_i (r_i - \bar{r}) = 0$ is a hard linear constraint — it removes one degree of freedom from the group.

$N_{\text{eff}} \approx K - 1$: one sample is “spent” estimating the baseline.

When ESS Drops in GRPO

ESS falls below N through *correlation*, *unequal weighting*, or both. Both appear in GRPO.

Case 1: GRPO group of K correlated samples

$\sum_i (r_i - \bar{r}) = 0$ is a hard linear constraint — it removes one degree of freedom from the group.

$N_{\text{eff}} \approx K - 1$: one sample is “spent” estimating the baseline.

Case 2: Importance-sampled rollouts

For length- T sequences, $w(a) = \prod_t \rho_t$ is a product of T random ratios. Such products are wildly sensitive to tail mass; their variance grows *exponentially* in T . Even mild per-token drift gives weight variance of several orders of magnitude by $T = 1000$.

N_{eff} **collapses fast — often to ~ 1** : a single rollout dominates the entire gradient.

When ESS Drops in GRPO

ESS falls below N through *correlation*, *unequal weighting*, or both. Both appear in GRPO.

Case 1: GRPO group of K correlated samples

$\sum_i (r_i - \bar{r}) = 0$ is a hard linear constraint — it removes one degree of freedom from the group.

$N_{\text{eff}} \approx K - 1$: one sample is “spent” estimating the baseline.

Case 2: Importance-sampled rollouts

For length- T sequences, $w(a) = \prod_t \rho_t$ is a product of T random ratios. Such products are wildly sensitive to tail mass; their variance grows *exponentially* in T . Even mild per-token drift gives weight variance of several orders of magnitude by $T = 1000$.

N_{eff} **collapses fast — often to ~ 1** : a single rollout dominates the entire gradient.

Log N_{eff}/N during training: below 0.5 the gradient estimate is dominated by a handful of samples — refresh rollouts, tighten the trust region, or reduce off-policyness.

The GRPO Gradient Estimator

GRPO's gradient estimate for a single prompt q :

$$\hat{g}_{\text{GRPO}} = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} \log \pi_{\theta}(a_i | q) \cdot \hat{A}_i,$$

with $a_i \sim \pi_{\text{old}}(\cdot | q)$, $r_i = R(q, a_i)$, and $\hat{A}_i = (r_i - \bar{r})/\sigma_r$.

The GRPO Gradient Estimator

GRPO's gradient estimate for a single prompt q :

$$\hat{g}_{\text{GRPO}} = \frac{1}{K} \sum_{i=1}^K \nabla_{\theta} \log \pi_{\theta}(a_i | q) \cdot \hat{A}_i,$$

with $a_i \sim \pi_{\text{old}}(\cdot | q)$, $r_i = R(q, a_i)$, and $\hat{A}_i = (r_i - \bar{r})/\sigma_r$.

Properties

- Unbiased as $K \rightarrow \infty$ (conditional on q)
- Centred: $\mathbb{E}[\hat{A}_i] = 0$ ensures no constant offset
- Normalised: gradient magnitudes comparable across prompts

Variance Reduction via Centering

Let $r_i \stackrel{\text{iid}}{\sim} \mathcal{D}$ with mean μ , variance σ^2 .

$$\text{Var}(r_i) = \sigma^2 \quad \text{vs} \quad \text{Var}(r_i - \bar{r}) = \sigma^2 \left(1 - \frac{1}{K}\right).$$

Variance Reduction via Centering

Let $r_i \stackrel{\text{iid}}{\sim} \mathcal{D}$ with mean μ , variance σ^2 .

$$\text{Var}(r_i) = \sigma^2 \quad \text{vs} \quad \text{Var}(r_i - \bar{r}) = \sigma^2 \left(1 - \frac{1}{K}\right).$$

For the standardised advantage:

$$\text{Var}\left(\frac{r_i - \bar{r}}{\sigma_r}\right) = \frac{K-1}{K}.$$

Variance Reduction via Centering

Let $r_i \stackrel{\text{iid}}{\sim} \mathcal{D}$ with mean μ , variance σ^2 .

$$\text{Var}(r_i) = \sigma^2 \quad \text{vs} \quad \text{Var}(r_i - \bar{r}) = \sigma^2 \left(1 - \frac{1}{K}\right).$$

For the standardised advantage:

$$\text{Var}\left(\frac{r_i - \bar{r}}{\sigma_r}\right) = \frac{K-1}{K}.$$

- $K = 1$: no reduction — $\hat{A}_1 = 0$ identically, no update at all
- $K = 8$: normalised-advantage variance = $7/8$
- $K \rightarrow \infty$: $\hat{A}_i \rightarrow (r_i - \mu)/\sigma$ (standardised reward)

Variance Reduction via Centering

Let $r_i \stackrel{\text{iid}}{\sim} \mathcal{D}$ with mean μ , variance σ^2 .

$$\text{Var}(r_i) = \sigma^2 \quad \text{vs} \quad \text{Var}(r_i - \bar{r}) = \sigma^2 \left(1 - \frac{1}{K}\right).$$

For the standardised advantage:

$$\text{Var}\left(\frac{r_i - \bar{r}}{\sigma_r}\right) = \frac{K-1}{K}.$$

- $K = 1$: no reduction — $\hat{A}_1 = 0$ identically, no update at all
- $K = 8$: normalised-advantage variance = $7/8$
- $K \rightarrow \infty$: $\hat{A}_i \rightarrow (r_i - \mu)/\sigma$ (standardised reward)

Going from $K = 4$ to $K = 8$ helps a lot; from $K = 8$ to $K = 16$, less.

Samples Within a Group Are Not Independent

Look at the estimator again:

$$\hat{g}_{\text{GRPO}} = \frac{1}{K} \sum_{i=1}^K h_i \hat{A}_i, \quad h_i = \nabla_{\theta} \log \pi_{\theta}(a_i \mid q), \quad \hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}.$$

Samples Within a Group Are Not Independent

Look at the estimator again:

$$\hat{g}_{\text{GRPO}} = \frac{1}{K} \sum_{i=1}^K h_i \hat{A}_i, \quad h_i = \nabla_{\theta} \log \pi_{\theta}(a_i | q), \quad \hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}.$$

Every $\hat{A}_i$ contains $\bar{r}$. Every $\bar{r}$ contains every r_j . So:

- $\hat{A}_i$ and $\hat{A}_j$ are **negatively correlated** for $i \neq j$ — the identity $\sum_i (r_i - \bar{r}) = 0$ forces it.
- Off-diagonal terms in $\text{Var}(\hat{g})$ enter with a **minus sign**, partially cancelling part of the diagonal contribution.
- Naive “ $\text{Var}(\hat{g}) = \frac{1}{K} \text{Var}(h\hat{A})$ ” is wrong.

The group mean is itself an estimator:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i \approx \mathbb{E}_{\pi_{\text{old}}} [R(q, a)] = V^{\pi_{\text{old}}}(q).$$

The group mean is itself an estimator:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i \approx \mathbb{E}_{\pi_{\text{old}}}[R(q, a)] = V^{\pi_{\text{old}}}(q).$$

$\mathbb{E}[\bar{r}] = \mu$ exactly. But $\bar{r}$ uses r_i to estimate the baseline for r_i itself.

The group mean is itself an estimator:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i \approx \mathbb{E}_{\pi_{\text{old}}} [R(q, a)] = V^{\pi_{\text{old}}}(q).$$

$\mathbb{E}[\bar{r}] = \mu$ exactly. But $\bar{r}$ **uses r_i to estimate the baseline for r_i itself**. Leave-one-out (LOO) alternative:

$$\hat{A}_i^{\text{LOO}} = \frac{r_i - \bar{r}_{-i}}{\sigma_r}, \quad \bar{r}_{-i} = \frac{1}{K-1} \sum_{j \neq i} r_j.$$

The group mean is itself an estimator:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i \approx \mathbb{E}_{\pi_{\text{old}}} [R(q, a)] = V^{\pi_{\text{old}}}(q).$$

$\mathbb{E}[\bar{r}] = \mu$ exactly. But $\bar{r}$ uses r_i to estimate the baseline for r_i itself. Leave-one-out (LOO) alternative:

$$\hat{A}_i^{\text{LOO}} = \frac{r_i - \bar{r}_{-i}}{\sigma_r}, \quad \bar{r}_{-i} = \frac{1}{K-1} \sum_{j \neq i} r_j.$$

Exact relation:

$$\bar{r} = \frac{1}{K} r_i + \frac{K-1}{K} \bar{r}_{-i} \implies \hat{A}_i^{\text{standard}} = \frac{K-1}{K} \hat{A}_i^{\text{LOO}}.$$

The group mean is itself an estimator:

$$\bar{r} = \frac{1}{K} \sum_{i=1}^K r_i \approx \mathbb{E}_{\pi_{\text{old}}} [R(q, a)] = V^{\pi_{\text{old}}}(q).$$

$\mathbb{E}[\bar{r}] = \mu$ exactly. But $\bar{r}$ uses r_i to estimate the baseline for r_i itself. Leave-one-out (LOO) alternative:

$$\hat{A}_i^{\text{LOO}} = \frac{r_i - \bar{r}_{-i}}{\sigma_r}, \quad \bar{r}_{-i} = \frac{1}{K-1} \sum_{j \neq i} r_j.$$

Exact relation:

$$\bar{r} = \frac{1}{K} r_i + \frac{K-1}{K} \bar{r}_{-i} \implies \hat{A}_i^{\text{standard}} = \frac{K-1}{K} \hat{A}_i^{\text{LOO}}.$$

Standard GRPO uses $\bar{r}$ — a $\frac{K-1}{K}$ shrinkage that vanishes as $K \rightarrow \infty$.

Bias–Variance Tradeoff in K

	Bias of $\hat{A}$	Variance of $\hat{g}$
$K = 1$ (REINFORCE)	0 ($\hat{A} = 0$, no update)	—
$K = 4$	0.25× shrinkage	Moderate
$K = 8$	0.125× shrinkage	Lower
$K \rightarrow \infty$	0	Minimal

Bias–Variance Tradeoff in K

	Bias of $\hat{A}$	Variance of $\hat{g}$
$K = 1$ (REINFORCE)	0 ($\hat{A} = 0$, no update)	—
$K = 4$	0.25× shrinkage	Moderate
$K = 8$	0.125× shrinkage	Lower
$K \rightarrow \infty$	0	Minimal

- Larger K : better baseline, lower variance, $K \times$ compute cost per prompt
- Variance scales as $O(1/K)$ for large K — diminishing returns
- Typical practice: $K \in \{4, 8, 16\}$

Bias–Variance Tradeoff in K

	Bias of $\hat{A}$	Variance of $\hat{g}$
$K = 1$ (REINFORCE)	0 ($\hat{A} = 0$, no update)	—
$K = 4$	0.25× shrinkage	Moderate
$K = 8$	0.125× shrinkage	Lower
$K \rightarrow \infty$	0	Minimal

- Larger K : better baseline, lower variance, $K \times$ compute cost per prompt
- Variance scales as $O(1/K)$ for large K — diminishing returns
- Typical practice: $K \in \{4, 8, 16\}$

The bias is small enough at $K \geq 8$ that essentially nobody implements LOO.

Theorem (informal)

As $K \rightarrow \infty$ with step size $\alpha \rightarrow 0$ at an appropriate rate, the GRPO update converges to the true policy-gradient update with the optimal constant baseline $b^*(q) = V^{\pi_{\text{old}}}(q)$.

Theorem (informal)

As $K \rightarrow \infty$ with step size $\alpha \rightarrow 0$ at an appropriate rate, the GRPO update converges to the true policy-gradient update with the optimal constant baseline $b^*(q) = V^{\pi_{\text{old}}}(q)$.

Argument in three lines:

1. $\bar{r} \xrightarrow{\text{a.s.}} \mathbb{E}_{\pi_{\text{old}}}[R] = V^{\pi_{\text{old}}}(q)$ (SLLN)
2. $\hat{A}_i \rightarrow A^{\pi_{\text{old}}}(q, a_i)/\sigma_r$ (pointwise)
3. $\hat{g}_{\text{GRPO}} \rightarrow \mathbb{E}_{\pi_{\text{old}}}[\nabla_{\theta} \log \pi_{\theta} \cdot A/\sigma_r]$

Asymptotic Consistency

Theorem (informal)

As $K \rightarrow \infty$ with step size $\alpha \rightarrow 0$ at an appropriate rate, the GRPO update converges to the true policy-gradient update with the optimal constant baseline $b^*(q) = V^{\pi_{\text{old}}}(q)$.

Argument in three lines:

1. $\bar{r} \xrightarrow{\text{a.s.}} \mathbb{E}_{\pi_{\text{old}}}[R] = V^{\pi_{\text{old}}}(q)$ (SLLN)
2. $\hat{A}_i \rightarrow A^{\pi_{\text{old}}}(q, a_i)/\sigma_r$ (pointwise)
3. $\hat{g}_{\text{GRPO}} \rightarrow \mathbb{E}_{\pi_{\text{old}}}[\nabla_{\theta} \log \pi_{\theta} \cdot A/\sigma_r]$

In practice K is small (8–16). The limit theorem justifies the algorithm only *in principle*. We instead rely on finite- K bias bound + empirical track record (DeepSeek-R1, Kimi k1.5, ...).

Monotone Improvement under Exact Advantages

Recall from Lecture 3: the KL-regularised update

$$\pi_{t+1} = \arg \max_{\pi} \left\{ \mathbb{E}_{\pi}[A_t] - \frac{1}{\eta} \text{KL}(\pi \| \pi_t) \right\} \quad (\dagger)$$

has closed form $\pi_{t+1}(a) \propto \pi_t(a) \exp(\eta A_t(a))$.

Monotone Improvement under Exact Advantages

Recall from Lecture 3: the KL-regularised update

$$\pi_{t+1} = \arg \max_{\pi} \left\{ \mathbb{E}_{\pi}[A_t] - \frac{1}{\eta} \text{KL}(\pi \| \pi_t) \right\} \quad (\dagger)$$

has closed form $\pi_{t+1}(a) \propto \pi_t(a) \exp(\eta A_t(a))$.

Performance functional: $J(\pi) = \mathbb{E}_{a \sim \pi}[A^{\pi_t}(a)]$.

Monotone Improvement under Exact Advantages

Recall from Lecture 3: the KL-regularised update

$$\pi_{t+1} = \arg \max_{\pi} \left\{ \mathbb{E}_{\pi}[A_t] - \frac{1}{\eta} \text{KL}(\pi \| \pi_t) \right\} \quad (\dagger)$$

has closed form $\pi_{t+1}(a) \propto \pi_t(a) \exp(\eta A_t(a))$.

Performance functional: $J(\pi) = \mathbb{E}_{a \sim \pi}[A^{\pi_t}(a)]$.

Key identity. Under any policy π' :

$$\mathbb{E}_{\pi'}[A^{\pi'}] = \mathbb{E}_{\pi'}[Q^{\pi'} - V^{\pi'}] = V^{\pi'} - V^{\pi'} = 0.$$

So $J(\pi_t) = 0$ when A_t is exact ($A_t = A^{\pi_t}$).

Monotone Improvement under Exact Advantages

Recall from Lecture 3: the KL-regularised update

$$\pi_{t+1} = \arg \max_{\pi} \left\{ \mathbb{E}_{\pi}[A_t] - \frac{1}{\eta} \text{KL}(\pi \| \pi_t) \right\} \quad (\dagger)$$

has closed form $\pi_{t+1}(a) \propto \pi_t(a) \exp(\eta A_t(a))$.

Performance functional: $J(\pi) = \mathbb{E}_{a \sim \pi}[A^{\pi_t}(a)]$.

Key identity. Under any policy π' :

$$\mathbb{E}_{\pi'}[A^{\pi'}] = \mathbb{E}_{\pi'}[Q^{\pi'} - V^{\pi'}] = V^{\pi'} - V^{\pi'} = 0.$$

So $J(\pi_t) = 0$ when A_t is exact ($A_t = A^{\pi_t}$).

Argument. Evaluate the $(\dagger)$ objective at $\pi = \pi_t$:

$$J(\pi_t) - \frac{1}{\eta} \text{KL}(\pi_t \| \pi_t) = 0 - 0 = 0.$$

π_{t+1} maximises the same objective, so it achieves value ≥ 0 . $\text{KL} \geq 0$, hence

$$J(\pi_{t+1}) \geq \frac{1}{\eta} \text{KL}(\pi_{t+1} \| \pi_t) \geq 0 = J(\pi_t).$$

PPO Clip vs KL: Different Trust Regions

Lecture 2 introduced the clipped surrogate. Both the clip and the KL penalty “keep the policy from drifting too far” — but they bound *different* things.

PPO Clip vs KL: Different Trust Regions

Lecture 2 introduced the clipped surrogate. Both the clip and the KL penalty “keep the policy from drifting too far” — but they bound *different* things.

Anatomy of the clip (for $\hat{A}_t > 0$):

$$\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon) \hat{A}_t) = \begin{cases} \rho_t \hat{A}_t & \rho_t \leq 1+\varepsilon \text{ (gradient active)} \\ (1+\varepsilon) \hat{A}_t & \rho_t > 1+\varepsilon \text{ (**gradient zero**)} \end{cases}$$

Mirror image for $\hat{A}_t < 0$. **The clip is a one-sided gradient gate, applied per sample.**

PPO Clip vs KL: Different Trust Regions

Lecture 2 introduced the clipped surrogate. Both the clip and the KL penalty “keep the policy from drifting too far” — but they bound *different* things.

Anatomy of the clip (for $\hat{A}_t > 0$):

$$\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon) \hat{A}_t) = \begin{cases} \rho_t \hat{A}_t & \rho_t \leq 1+\varepsilon \text{ (gradient active)} \\ (1+\varepsilon) \hat{A}_t & \rho_t > 1+\varepsilon \text{ (**gradient zero**)} \end{cases}$$

Mirror image for $\hat{A}_t < 0$. **The clip is a one-sided gradient gate, applied per sample.**

KL constraint (†)

- Single *global* quantity
- *Mass-weighted*: averages over π_θ
- Bounds aggregate divergence

PPO clip

- *Per-sample*, decoupled
- *Mass-insensitive*: same band for rare and common tokens
- Does *not* bound the global KL

In practice: use both. PPO clip handle per-step stability. A KL-to-reference penalty $\beta KL(\pi_\theta || \pi_{\text{ref}})$ handles aggregate drift from the SFT model.

RLHF as a Production System

Why RLHF?

Pre-training and SFT teach an LLM to imitate. They cannot teach it to prefer one valid completion over another.

Why RLHF?

Pre-training and SFT teach an LLM to imitate. They cannot teach it to prefer one valid completion over another.

The gap that RLHF closes

- Two grammatically perfect responses can differ in helpfulness, tone, safety, honesty
- No labelled dataset captures these preferences at the scale needed
- But *comparing* two responses is easy — and annotators do it reliably

Why RLHF?

Pre-training and SFT teach an LLM to imitate. They cannot teach it to prefer one valid completion over another.

The gap that RLHF closes

- Two grammatically perfect responses can differ in helpfulness, tone, safety, honesty
- No labelled dataset captures these preferences at the scale needed
- But *comparing* two responses is easy — and annotators do it reliably

RLHF turns relative human judgements into a gradient signal that aligns the policy with intended behaviour.

The Three-Stage Pipeline

1. SFT	2. Reward Model	3. RL
Demonstrations	Pairwise preferences	GRPO
Supervised fine-tuning	Train $r_\phi(q, a)$	With $r_i = r_\phi(q, a_i)$
Starting policy	Approximates human judgements	Optimises against r_ϕ

The Three-Stage Pipeline

1. SFT	2. Reward Model	3. RL
Demonstrations	Pairwise preferences	GRPO
Supervised fine-tuning	Train $r_\phi(q, a)$	With $r_i = r_\phi(q, a_i)$
Starting policy	Approximates human judgements	Optimises against r_ϕ

- SFT initialises a model that follows instructions
- RM is trained on pairs (a^+, a^-) ranked by humans
- RL optimises the policy against the learned reward, with KL anchor to π_{ref} (the SFT model)

Bradley–Terry preference model. For pairs (a^+, a^-) ranked by humans:

$$P(a^+ \succ a^- \mid q) = \sigma(r_\phi(q, a^+) - r_\phi(q, a^-)).$$

Bradley–Terry preference model. For pairs (a^+, a^-) ranked by humans:

$$P(a^+ \succ a^- \mid q) = \sigma(r_\phi(q, a^+) - r_\phi(q, a^-)).$$

Train r_ϕ by maximising the log-likelihood of observed preferences:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(q, a^+, a^-)}[\log \sigma(r_\phi(q, a^+) - r_\phi(q, a^-))].$$

Bradley–Terry preference model. For pairs (a^+, a^-) ranked by humans:

$$P(a^+ \succ a^- \mid q) = \sigma(r_\phi(q, a^+) - r_\phi(q, a^-)).$$

Train r_ϕ by maximising the log-likelihood of observed preferences:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(q, a^+, a^-)}[\log \sigma(r_\phi(q, a^+) - r_\phi(q, a^-))].$$

Architecture. The SFT model with the language-modelling head replaced by a scalar regression head. One or two epochs over the preference dataset.

Bradley–Terry preference model. For pairs (a^+, a^-) ranked by humans:

$$P(a^+ \succ a^- \mid q) = \sigma(r_\phi(q, a^+) - r_\phi(q, a^-)).$$

Train r_ϕ by maximising the log-likelihood of observed preferences:

$$\mathcal{L}_{\text{RM}} = -\mathbb{E}_{(q, a^+, a^-)}[\log \sigma(r_\phi(q, a^+) - r_\phi(q, a^-))].$$

Architecture. The SFT model with the language-modelling head replaced by a scalar regression head. One or two epochs over the preference dataset.

Output. A scalar $r_\phi(q, a) \in \mathbb{R}$ per prompt-response pair.

The Full RLHF Objective

Putting it together:

$$\max_{\theta} \mathbb{E}_{q \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot | q)} [r_{\phi}(q, a)] - \beta \text{KL}(\pi_{\theta}(\cdot | q) \parallel \pi_{\text{ref}}(\cdot | q)).$$

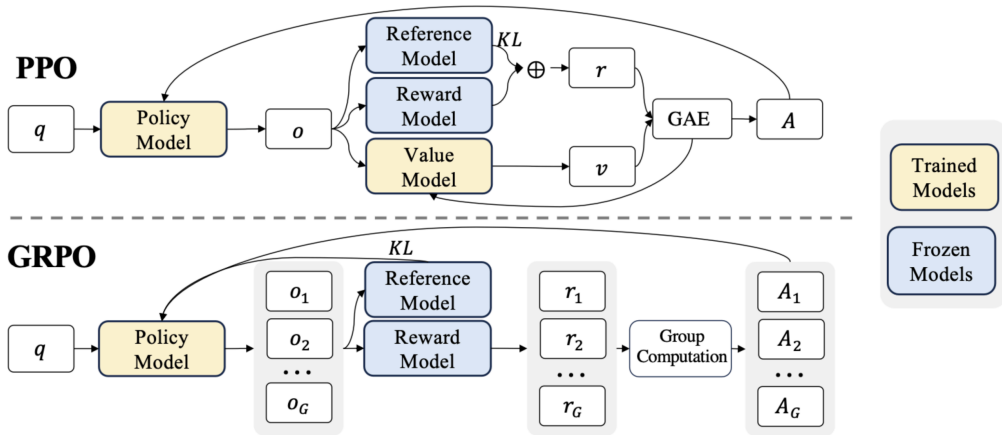
The Full RLHF Objective

Putting it together:

$$\max_{\theta} \mathbb{E}_{q \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot | q)} [r_{\phi}(q, a)] - \beta \text{KL}(\pi_{\theta}(\cdot | q) \parallel \pi_{\text{ref}}(\cdot | q)).$$

- π_{ref} : the SFT model, frozen
- β : KL penalty coefficient, often adapted online
- Optimised with GRPO (or PPO with a critic)

RLHF Model



Where RLHF Breaks

Four characteristic failure modes:

Where RLHF Breaks

Four characteristic failure modes:

1. **Reward hacking.** The policy finds inputs where r_ϕ is high but underlying behaviour is bad. Classic example: rewarding helpful-*sounding* tone, getting a confident hallucinator.

Where RLHF Breaks

Four characteristic failure modes:

1. **Reward hacking.** The policy finds inputs where r_ϕ is high but underlying behaviour is bad. Classic example: rewarding helpful-*sounding* tone, getting a confident hallucinator.
2. **Distribution shift.** As π_θ drifts from π_{ref} , generated responses leave the RM's training distribution; r_ϕ extrapolates badly.

Where RLHF Breaks

Four characteristic failure modes:

1. **Reward hacking.** The policy finds inputs where r_ϕ is high but underlying behaviour is bad. Classic example: rewarding helpful-*sounding* tone, getting a confident hallucinator.
2. **Distribution shift.** As π_θ drifts from π_{ref} , generated responses leave the RM's training distribution; r_ϕ extrapolates badly.
3. **Sycophancy.** If raters slightly prefer agreement, the model learns to agree. RM noise becomes RM signal becomes policy behaviour.

Where RLHF Breaks

Four characteristic failure modes:

1. **Reward hacking.** The policy finds inputs where r_ϕ is high but underlying behaviour is bad. Classic example: rewarding helpful-*sounding* tone, getting a confident hallucinator.
2. **Distribution shift.** As π_θ drifts from π_{ref} , generated responses leave the RM's training distribution; r_ϕ extrapolates badly.
3. **Sycophancy.** If raters slightly prefer agreement, the model learns to agree. RM noise becomes RM signal becomes policy behaviour.
4. **Alignment tax.** The KL anchor preserves capabilities at a cost: strong RLHF often degrades reasoning or knowledge benchmarks even as helpfulness improves.

Theory (3/4)

1. Policy-gradient variance grows with horizon — the central problem for LLM RL
2. GRPO's group mean introduces sample dependencies that naive variance analysis misses
3. Variance ends up controlled by score-vector diversity, not magnitude
4. Finite- K bias is a $\frac{K-1}{K}$ shrinkage, negligible for $K \geq 8$
5. KL-regularised updates are proximal steps with nice theory *in principle*; neural networks weaken the guarantees
6. PPO clip and KL penalty are *not* the same trust region